Introduction to Computer Systems

University Lecture Note

Lecture 3: Memory Hierarchy

A comprehensive note synthesized from lecture slides, the lecture transcript, and the uploaded study materials

Course instructors: Wei Xu, Mingyu Gao
Teaching assistants: Weitao Wan, Tingrui Zhang, Ruitao Liu, Shuo Chen

Spring 2026

IIIS, Tsinghua University

Contents

1 Introduction **1**

2 Memory in the Big Picture of Computer Systems **1**

2.1 From the Processor to Data Storage . . . 1

2.2 Primary Storage versus Secondary Storage . . . 1

2.3 Why the Separation Exists . . . 2

3 Memory Technologies and the Capacity-Latency Tradeoff **3**

3.1 SRAM: Fast but Small . . . 3

3.2 DRAM: Dense but Slower . . . 4

3.3 Comparing SRAM and DRAM . . . 5

3.4 The Fundamental Tradeoff . . . 5

3.5 The Processor-Memory Gap . . . 5

3.6 Emerging Non-Volatile Memories . . . 5

4 Program Locality and the Idea of a Memory Hierarchy **6**

4.1 Optimizing for the Common Case . . . 6

4.2 Temporal and Spatial Locality . . . 6

4.3 Locality in Data and Instructions . . . 7

4.4 When Locality Fails . . . 8

4.5 The Memory Hierarchy as an Architectural Opportunity . . . 8

4.6 Working Sets . . . 9

5 Processor Caches **9**

5.1 What a Cache Is . . . 9

5.2 Why Blocks Matter . . . 10

5.3 Quantifying Cache Performance . . . 10

5.4 Cache Lookup as an Address-Matching Problem . . . 11

5.5 Associativity: Fully Associative, Direct-Mapped, and Set-Associative . . . 11

5.5.1 Fully Associative Caches . . . 11

5.5.2 Direct-Mapped Caches . . . 11

5.5.3 Set-Associative Caches . . . 11

5.6 Tags and Valid Bits . . . 12

5.7 Worked Example: Address Decomposition . . . 13

5.8 The 3Cs of Cache Misses . . . 13

6 Replacement and Write Policies 13

6.1 Replacement: Which Block Becomes the Victim? . . . 13

6.2 Bypass and Insertion Choices . . . 14

6.3 Write Hits and Write Misses . . . 15

6.3.1 Write-Back versus Write-Through . . . 15

6.3.2 Write-Allocate versus No-Write-Allocate . . . 15

6.4 The "ABC" View of Cache Configuration . . . 15

7 Cache-Friendly Programming and Performance Optimization 16

7.1 Caches Are Functionally Transparent but Performance Visible . . . 16

7.2 The Memory Mountain . . . 16

7.3 Arrays, Row-Major Layout, and Stride . . . 17

7.4 A Simple Example: Summing a Matrix . . . 17

7.5 Matrix Multiplication as a Case Study . . . 17

7.6 Loop Reordering . . . 18

7.7 Loop Blocking (Tiling) . . . 19

7.8 Why Blocking Works . . . 20

7.9 Compiler and Library Support . . . 20

8 Summary 20

9 Quizzes & Exercises 20

9.1 Review Questions . . . 20

9.2 Analytical Exercises . . . 21

9.3 Programming-Oriented Exercises . . . 21

9.4 Optional Preview Questions from the Uploaded Auxiliary Guide . . . 22

1 Introduction

Computer systems are organized around an apparently simple question: *where do data live while computation is taking place?* In the abstract von Neumann view, a processor executes instructions and a memory system stores both instructions and data. In practice, however, "memory" is not one uniform object. A modern machine contains registers, several cache levels, main memory, solid-state storage, disks, and often remote storage. These technologies differ dramatically in speed, capacity, cost, energy, and persistence.

This lecture studies the architectural idea that reconciles those differences: the *memory hierarchy*. The hierarchy is one of the foundational ideas in computer systems because it shows how a system can appear both large and fast even though no underlying storage technology is simultaneously large, cheap, and fast. The basic strategy is to exploit patterns in typical programs rather than optimize for the worst conceivable access pattern. Those patterns are called *locality*, and they explain why tiny on-chip memories can hide the latency of much larger off-chip memories.

The lecture has three intertwined goals. First, it explains why primary memory and secondary storage play different roles in a system. Second, it develops the hardware organization of caches, including hits and misses, associativity, tags, replacement, and write handling. Third, it connects architecture to programming practice by showing that source code structure—especially loop order, stride, and blocking—can strongly influence performance even when the algorithmic complexity is unchanged.

Definition

The **memory hierarchy** is a layered organization of storage in which each faster, smaller level holds a subset of the data from the next slower, larger level. If programs exhibit locality, most accesses are served by upper levels, so the overall system behaves much closer to the top of the hierarchy than to the bottom.

2 Memory in the Big Picture of Computer Systems

2.1 From the Processor to Data Storage

Earlier lectures emphasized the processor: instruction execution, datapaths, control, and the role of instruction-set architecture. Memory shifts the focus from *how computation is performed* to *how operands and instructions are supplied*. This shift is not a change of topic so much as a change of perspective. A fast processor without a correspondingly effective memory system quickly becomes starved for data.

In a classic von Neumann organization, the processor and memory are logically distinct. The processor contains control logic and compute units; memory stores program instructions and program data. This separation is conceptually useful because it distinguishes transformation from storage, but it also creates a bottleneck: every arithmetic operation ultimately depends on data being fetched from some form of storage.

2.2 Primary Storage versus Secondary Storage

At the highest level, storage in a computer system is divided into two broad classes.

Table 1: Contrasting primary storage and secondary storage.

Property	**Primary storage (main memory)**	**Secondary storage (external storage)**
Persistence	Volatile; contents are lost when power is removed	Non-volatile; contents survive power loss
Typical role	Temporary working state during program execution	Long-term storage of inputs, outputs, files, and system state
Access path	Accessed by hardware through load and store instructions	Accessed through the operating system and file system
Implementation	DRAM, assisted by SRAM caches	SSDs, disks, and related storage devices
Performance goal	Low latency for frequent accesses	Durability, capacity, and cost-efficiency
Sharing model	Typically private address spaces per process	Shared file formats visible across programs

Main memory is the processor's immediate working area. It holds stack frames, heaps, global data, code pages, and other transient state needed by a running program. Because these data are accessed frequently and fine-grained accesses must be fast, the hardware exposes memory directly through loads and stores. Secondary storage serves a different purpose: it preserves data beyond one execution and enables programs to exchange information through files or other persistent objects.

The distinction is familiar from everyday computing. Editing a document changes in-memory state first. Unless the user saves the file, a power failure can destroy those edits because they were never committed to persistent storage. This simple observation captures why the system intentionally separates fast temporary storage from slower durable storage.

2.3 Why the Separation Exists

The separation between primary and secondary storage is not arbitrary. It reflects competing engineering objectives.

First, program execution demands speed, not permanence. Intermediate variables, partially computed results, temporary buffers, and stack-resident state are valuable only while a computation is underway. It is therefore reasonable to store them in a technology optimized for latency and bandwidth, even if the contents disappear on power loss.

Second, persistent data require more than raw hardware state. Reliability, consistency, and recovery often depend on software techniques such as buffering policies, journaling, copy-on-write, checksums, logging, and transactions. Hardware alone usually cannot guarantee that a complex multi-step update remains valid in the presence of crashes or interruptions. This is why later systems topics such as databases and transaction processing build on software mechanisms rather than rely solely on device physics.

Third, each running program benefits from having a private in-memory representation tailored to its own computations, while persistent storage must usually use universal formats that support sharing and interoperability. One program's output file may be another program's input file, but each program is free to organize its internal in-memory structures for performance.

Deep Dive

The storage hierarchy embodies a broader systems principle: **speed, persistence, and sharing are different design goals**. Memory emphasizes speed for private, temporary state. Storage emphasizes persistence and interoperability. Attempting to optimize all three goals equally with one technology tends to fail because the mechanisms that make accesses fast are often different from the mechanisms that make updates durable and recoverable.

3 Memory Technologies and the Capacity-Latency Tradeoff

3.1 SRAM: Fast but Small

Static random access memory (SRAM) is the technology used for processor caches. "Random access" means that the system can directly read or write any addressable location without scanning through preceding locations. Physically, an SRAM array is typically organized as a two-dimensional structure with row and column decoders. An address is split into a row component and a column component; their intersection selects the storage cell to be read or written.

The word *static* indicates that a bit is represented by a stable circuit state, usually implemented with cross-coupled inverters. A standard SRAM cell uses six transistors, often called a *6T* cell. Because each bit requires several transistors, SRAM cells are relatively large, which limits density. The reward is speed: SRAM uses a transistor-centric technology similar to logic circuits and can therefore be placed on the same chip as the processor.

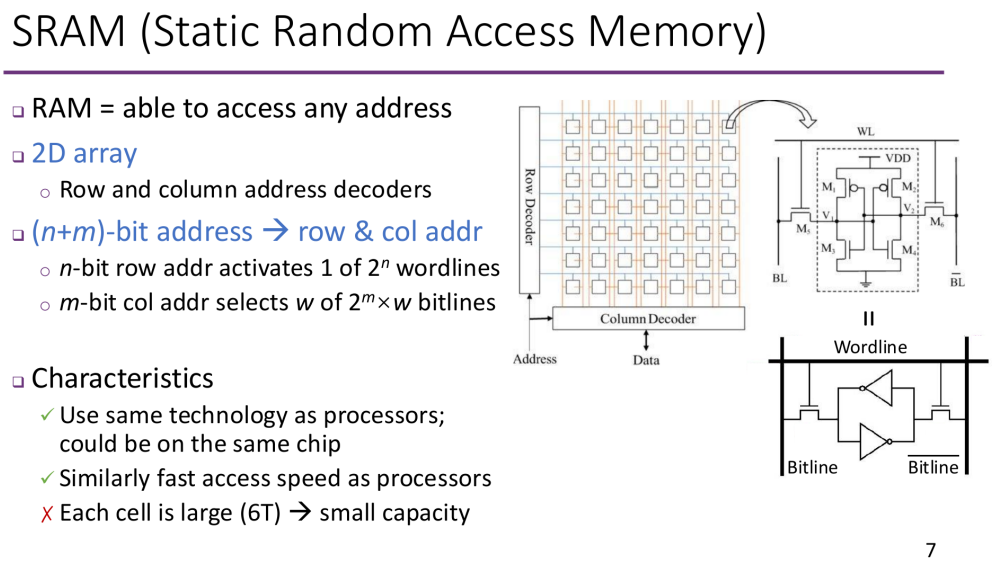


Figure 1: Structure of an SRAM array and a six-transistor SRAM cell. The slide illustrates row and column decoding, wordlines and bitlines, and the positive-feedback circuit used to store a stable bit. SRAM matches processor fabrication technology well and offers very low access latency, but its large cell area makes it expensive in capacity terms.

The performance implications are direct. Since the cell state is strong and local, SRAM supports short access latency and high bandwidth. Since each bit consumes significant area, the total

number of bits that can be integrated economically on-chip is modest. Hence SRAM naturally appears near the top of the hierarchy, where speed matters more than bulk capacity.

3.2 DRAM: Dense but Slower

Dynamic random access memory (DRAM) is the dominant technology for main memory. Like SRAM, it is typically arranged as a two-dimensional array addressed by rows and columns. The crucial difference lies in the storage cell. A conventional DRAM cell stores a bit as charge on a tiny capacitor, gated by a transistor. This is often summarized as a *1T1C* cell: one transistor and one capacitor.

Because the cell uses far fewer active devices than SRAM, DRAM is much denser. More bits fit into the same physical area, which makes DRAM well suited for gigabyte-scale main memory. The cost of that density is slower, more delicate access. Reading a DRAM cell involves sensing a very small amount of charge, and the read is effectively destructive: after the cell is sensed, its content must be restored. In addition, charge leaks over time, so DRAM must be periodically refreshed even if software does not access it.

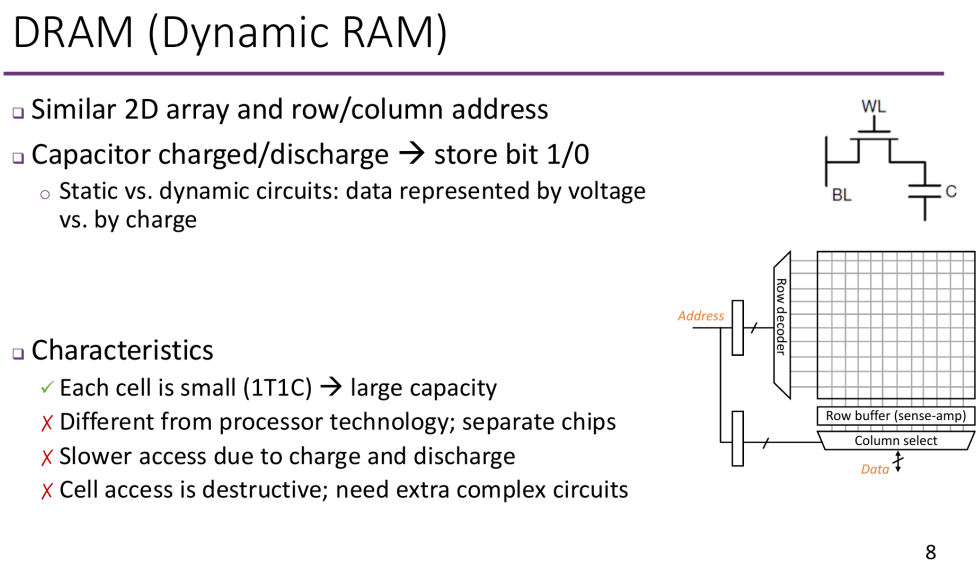


Figure 2: Organization of a DRAM array and the one-transistor, one-capacitor cell used to store a dynamic bit. The row decoder activates a row, a row buffer or sense amplifier captures and restores the fragile cell contents, and a column selection network chooses the desired output. The figure emphasizes why DRAM is denser than SRAM but also slower and more complex to access.

A particularly important DRAM structure is the *row buffer*. When a row is activated, the entire row is sensed into a temporary buffer. Subsequent accesses to nearby data in that same row can be served more efficiently than accesses that require a different row activation. This creates a form of locality within DRAM itself: row-buffer hits are cheaper than row-buffer misses.

3.3 Comparing SRAM and DRAM

Table 2: A high-level comparison of SRAM and DRAM.

Dimension	**SRAM**	**DRAM**
Storage mechanism	Stable transistor circuit (typically 6T)	Charge on capacitor plus access transistor (1T1C)
Density	Low	High
Latency	Very low	Higher
Typical placement	On-chip caches	Off-chip main memory
Read behavior	Non-destructive in normal operation	Destructive; data must be restored
Refresh required	No	Yes
Cost per bit	High	Lower

These technologies therefore occupy different layers of the hierarchy for structural reasons, not merely historical convention.

3.4 The Fundamental Tradeoff

No practical memory technology simultaneously delivers the best latency, the best density, the lowest energy per useful operation, and the lowest cost per bit. Larger arrays take longer to traverse electrically, denser cells are usually harder to drive strongly, and persistent technologies often involve slower physical state changes than volatile ones.

At a coarse level, the hierarchy progresses from SRAM to DRAM to solid-state storage to magnetic disks and then, in distributed environments, to network-attached or remote storage. Each downward step generally increases capacity and reduces cost per bit, but also increases latency dramatically. This tradeoff is not a temporary implementation accident; it reflects deep physical constraints.

3.5 The Processor-Memory Gap

An additional problem makes raw memory latency even more severe: processor performance has historically improved faster than main-memory latency. Even when both improve, the *relative* gap can widen. As a result, a processor capable of executing several operations per cycle may be stalled for tens or hundreds of cycles waiting for a single main-memory access.

This mismatch is one of the central motivations for modern computer architecture. A high-performance core is useful only if the memory system can keep its execution units supplied with instructions and operands. Memory hierarchy techniques address exactly this bottleneck.

3.6 Emerging Non-Volatile Memories

The traditional contrast between volatile memory and non-volatile storage has softened somewhat with the development of phase-change memory (PCM), spin-transfer torque RAM (STT-RAM), resistive RAM (ReRAM), and related technologies. These memories store data using physical states such as material phase, magnetic orientation, or resistance levels. They can offer persistence

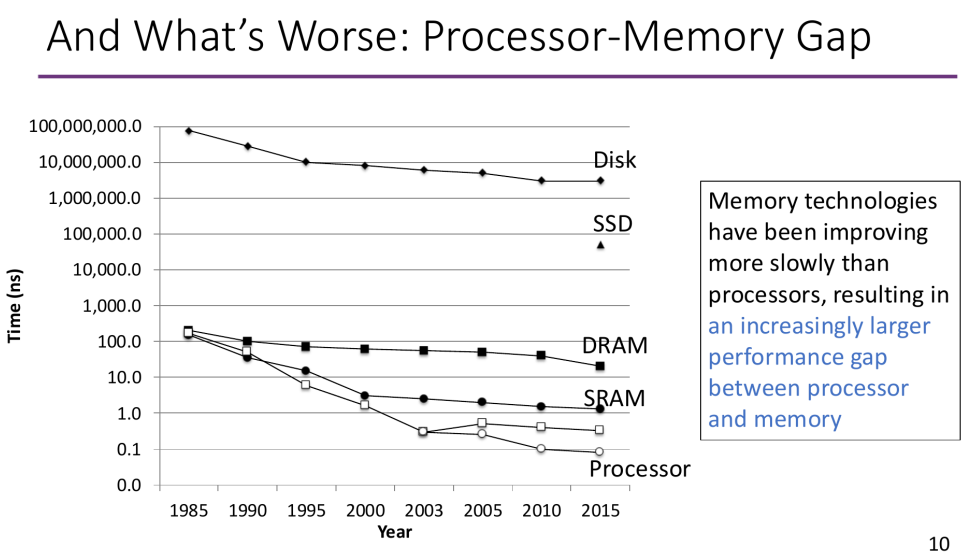


Figure 3: Historical latency trends for processor and storage technologies. The graph shows that processor latency improved faster than the latency of DRAM and long-term storage, producing a widening processor-memory gap. This divergence explains why architectural techniques such as caching, prefetching, and latency hiding became essential rather than optional.

and potentially higher density than DRAM, though often with higher access latency, asymmetric read/write cost, or endurance concerns.

These technologies do not eliminate the hierarchy. Instead, they enrich it by adding additional intermediate points in the design space.

4 Program Locality and the Idea of a Memory Hierarchy

4.1 Optimizing for the Common Case

A recurring lesson in systems design is that the common case deserves disproportionate attention. A memory system need not make every access uniformly fast. It must make *typical* access patterns fast. To do that, the system first needs a model of what typical programs do.

The crucial observation is that most programs do not touch all of their data uniformly at every moment. Instead, execution tends to move through phases. At any given time, only a relatively small subset of code and data is active. If a system can keep that active subset near the processor, it can behave much faster than a monolithic large memory would suggest.

4.2 Temporal and Spatial Locality

Definition

Temporal locality means that if a location is referenced, it is likely to be referenced again soon. **Spatial locality** means that if a location is referenced, nearby locations are likely to be referenced soon.

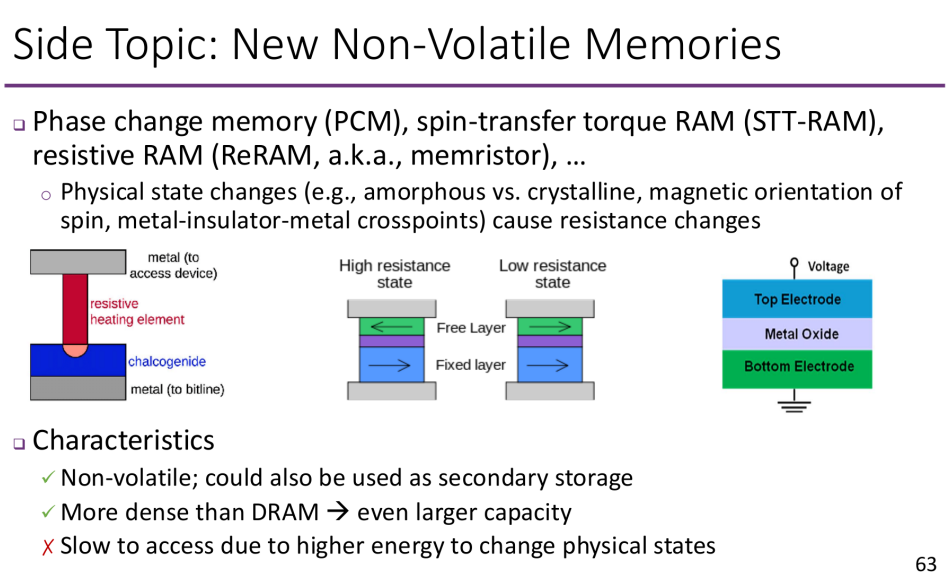


Figure 4: Examples of emerging non-volatile memory technologies. The slide sketches representative device mechanisms for phase-change memory, magnetic RAM, and resistive RAM, highlighting the ongoing search for technologies that may partially bridge the gap between memory and storage while introducing new design tradeoffs of their own.

Temporal locality often appears when a value is reused in a short time window. Loop indices, running sums, accumulator variables, stack variables, recently called functions, and repeatedly tested flags are common examples. Spatial locality appears when programs traverse contiguous memory, such as arrays, sequential instruction streams, stack frames, or adjacent structure fields.

The lecture transcript emphasizes an important qualitative habit: a professional systems programmer should be able to inspect code and quickly form an intuition about its locality. That intuition often predicts performance far better than asymptotic complexity alone.

4.3 Locality in Data and Instructions

Both *data* and *instructions* exhibit locality.

For data, array traversals are the classic example of spatial locality. In C and C++, if an array is stored contiguously, then referencing `a[i]` often implies that `a[i+1]` and `a[i+2]` will soon be needed as well. Scalar variables used inside a loop body typically exhibit temporal locality because the same memory locations are accessed across many iterations.

For instructions, the sequential nature of instruction fetch produces spatial locality almost automatically: once the processor fetches one instruction, it will often fetch the next few instructions from nearby addresses. Loops produce temporal locality in instruction references because the same body is executed repeatedly.

Deep Dive

Instruction locality matters just as much as data locality. This is why processors typically include separate L1 instruction and data caches. Code with tiny hot loops can fit entirely inside the instruction cache, enabling very high fetch efficiency. Conversely, extremely large code footprints or heavy use of indirect branches can degrade instruction-side behavior even when data locality is good.

4.4 When Locality Fails

Not all programs are cache-friendly. Pointer chasing in linked lists, trees, graphs, and hash-based structures can destroy spatial locality because each pointer may lead to a distant memory location. If the traversal is one-time and non-repeating, temporal locality can be weak as well.

This leads to a subtle but important systems lesson: *algorithmic efficiency and machine efficiency are not identical*. A linked structure may offer attractive asymptotic insertion or deletion costs, but an array-based representation may still run faster for realistic sizes because it exploits the hardware better. In systems work, constant factors created by the memory hierarchy are often large enough to dominate performance.

4.5 The Memory Hierarchy as an Architectural Opportunity

If fast memories are small and typical programs repeatedly access small active subsets, then those two facts complement each other. The memory hierarchy stores all data in a large lower level, but keeps the most recently and most nearby accessed pieces in smaller, faster upper levels. Each level caches a subset of the next level below it.

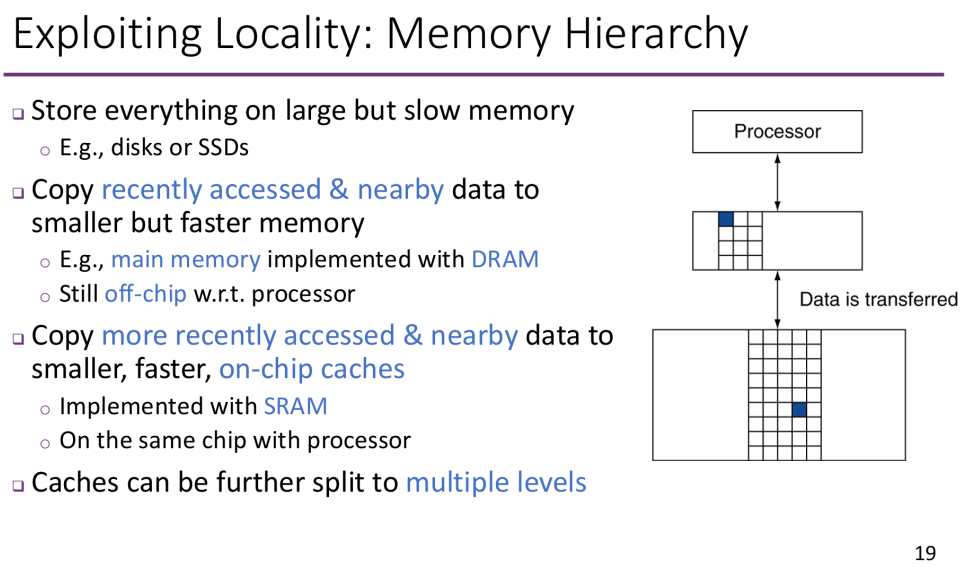


Figure 5: The conceptual movement of data through a hierarchy. Recently used and nearby data are copied upward from large slow storage into DRAM and then further upward into on-chip SRAM caches close to the processor. Each transfer is justified by expected reuse, not by the hope that every byte will be equally valuable.

The hierarchy succeeds because it turns the unevenness of program behavior into an advantage.

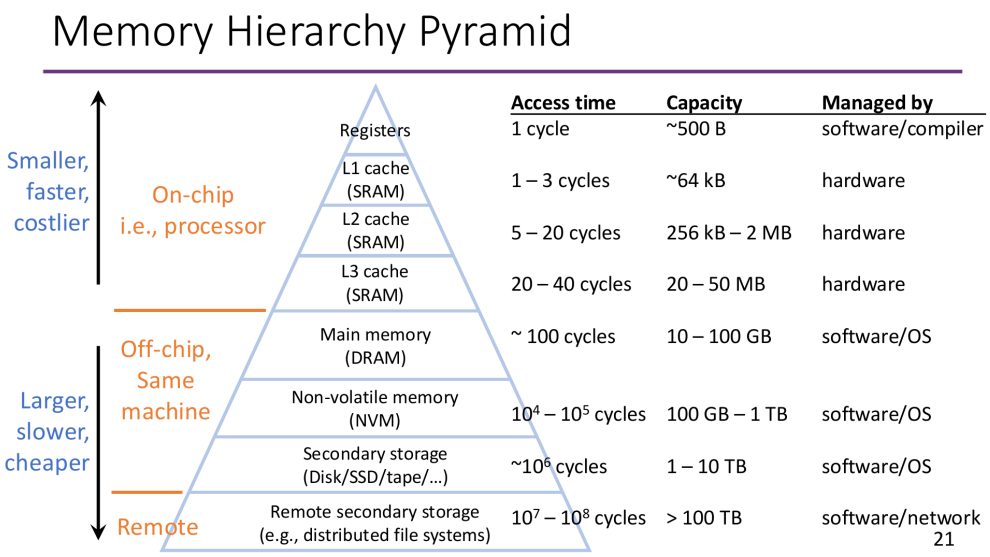


Figure 6: The memory hierarchy pyramid, from registers and on-chip caches down to main memory, non-volatile memory, secondary storage, and remote storage. The figure emphasizes the monotonic trend: upper levels are smaller, faster, and more expensive per bit, while lower levels are larger, slower, and cheaper. Management responsibility also shifts from hardware near the top to the operating system and distributed software near the bottom.

The system can deliver performance close to that of the top level on most accesses, while paying a cost closer to that of the bottom level for total capacity.

4.6 Working Sets

A useful mental model is the *working set*: the collection of instructions and data that a program phase is likely to reuse soon. When the working set fits in a given cache level, performance is usually excellent. When it exceeds that level, miss rates rise and the next lower level becomes important. This concept connects algorithm design directly to hardware behavior: changing loop structure can change the effective working set and therefore change performance dramatically.

5 Processor Caches

5.1 What a Cache Is

A cache is a small, fast memory placed near the processor to hold recently used blocks of instructions or data from a larger memory. From the software point of view, the processor still issues ordinary load and store instructions to ordinary addresses. The cache is managed by hardware and is mostly invisible to correctness. What changes is latency.

Because entire memory systems are too large to search naively on every access, caches move data in *blocks*, also called *cache lines*. A block is the unit of transfer between adjacent levels. If one byte from a 64-byte block is requested and the block is absent, the hardware usually fetches the whole 64-byte block, anticipating spatial locality.

Definition

A **cache hit** occurs when the requested address is found in the cache and can be served immediately from that level. A **cache miss** occurs when the requested block is absent and must be fetched from a lower level, potentially evicting an older block.

5.2 Why Blocks Matter

Block-based transfer is a direct architectural bet on spatial locality. If programs often access nearby addresses, then the cost of fetching several adjacent bytes at once is worthwhile because future requests will reuse the fetched block. Larger blocks can reduce compulsory misses, but they also consume more cache capacity and may waste bandwidth if only a tiny portion of the block is used. Block size is therefore one of the core cache design parameters.

5.3 Quantifying Cache Performance

The standard first-order model of cache performance uses four quantities.

- **Hit rate**: the fraction of accesses that hit.
- **Miss rate**: the fraction of accesses that miss, equal to $1 -$ hit rate.
- **Hit latency**: the time to access the cache when the data are present.
- **Miss penalty**: the additional delay incurred to fetch the block from a lower level and make the access complete.

These combine into the key metric:

$$\text{AMAT} = \text{hit latency} + \text{miss rate} \times \text{miss penalty}.$$

Definition

Average Memory Access Time (AMAT) is the expected latency of a memory reference, averaged over hits and misses. It is often the most useful compact summary of memory-system performance.

A crucial lesson is that a small improvement in miss rate can matter enormously because miss penalties are so much larger than hit times. If hit time is 1 ns and miss penalty is 100 ns, then improving hit rate from 97% to 99% changes

$$\text{AMAT}_{97\%} = 1 + 0.03 \times 100 = 4\,\text{ns},$$

while

$$\text{AMAT}_{99\%} = 1 + 0.01 \times 100 = 2\,\text{ns}.$$

The hit rate improved by only two percentage points, but the average access time improved by a factor of two.

5.4 Cache Lookup as an Address-Matching Problem

A cache must answer two questions for each access: where should the relevant block be searched for, and how can the hardware tell whether the block currently stored there is the correct one? The answer introduces the ideas of sets, tags, valid bits, and associativity.

Suppose a cache stores blocks of size B bytes, has data capacity C bytes, and associativity A. Then the number of sets is

$$S = \frac{C}{A\,B}.$$

If addresses are byte addresses, then:

$$\text{block address} = \left\lfloor \frac{\text{address}}{B} \right\rfloor,$$

$$\text{set index} = \text{block address} \bmod S,$$

$$\text{tag} = \left\lfloor \frac{\text{block address}}{S} \right\rfloor.$$

The address bits are therefore split into:

- **offset bits**: $\log_2 B$,
- **index bits**: $\log_2 S = \log_2\left(\frac{C}{AB}\right)$,
- **tag bits**: the remaining upper address bits.

5.5 Associativity: Fully Associative, Direct-Mapped, and Set-Associative

5.5.1 Fully Associative Caches

In a fully associative cache, a block may be placed anywhere. This minimizes placement restrictions and therefore reduces conflict misses. However, the hardware must compare the incoming tag against all entries, which increases lookup complexity and usually hit latency. Fully associative designs are therefore most practical for very small structures, such as translation lookaside buffers or victim caches, rather than large data caches.

5.5.2 Direct-Mapped Caches

In a direct-mapped cache, each block has exactly one possible location. The set index fully determines the line. This makes lookup simple and fast, since only one candidate entry must be checked. The downside is greater susceptibility to conflicts: two unrelated blocks that map to the same line can continually evict one another even if most of the cache is unused.

The direct-mapped index is often taken from lower block-address bits because the number of sets is usually a power of two. Then modulo arithmetic becomes a simple bit extraction. This is hardware-friendly and naturally distributes consecutive blocks across consecutive sets.

5.5.3 Set-Associative Caches

An A-way set-associative cache strikes a middle ground. Each block maps to exactly one set, but within that set it may occupy any of A ways. Higher associativity reduces conflicts but requires more tag comparisons and more metadata. In practice, modest associativity—for example 2-way, 4-way, or 8-way—often captures most of the benefit without the full cost of full associativity.

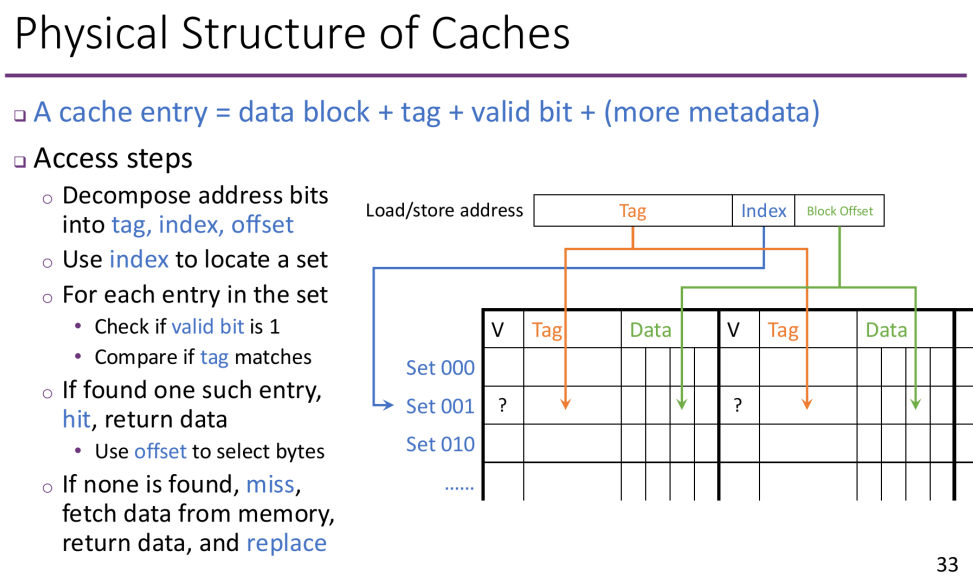


Figure 7: Physical organization of a set-associative cache. An incoming address is divided into tag, index, and block offset. The index selects a set, the tags of all ways in that set are compared in parallel, valid bits identify occupied entries, and the block offset selects the desired byte or word from the matched cache line. On a miss, the cache fetches the requested block from a lower level and replaces some entry in the indexed set.

5.6 Tags and Valid Bits

A cache line stores more than data. It also stores metadata that tell the hardware whether the line is meaningful and which memory block it represents.

- The **valid bit** indicates whether the entry contains a usable block.
- The **tag** records the upper address bits needed to distinguish among the many memory blocks that could map to the same set.

The index bits do not need to be stored because the set is already known from where the line resides physically. The block offset does not need to be stored because it identifies a byte within the block, not which block the cache line represents.

5.7 Worked Example: Address Decomposition

Worked Example

A cache has 1024 entries, a block size of 64 bytes, and 48-bit physical addresses.
Data capacity. Since each entry holds one 64-byte block, the data capacity is

$$1024 \times 64 = 65536 \text{ bytes} = 64 \text{ KiB}.$$

Offset bits. Since $64 = 2^6$, the block offset needs 6 bits.
Fully associative. There is one set, so index bits = 0 and tag bits = $48 - 6 = 42$.
Direct-mapped. There are 1024 sets, so index bits = $\log_2 1024 = 10$. Tag bits = $48 - 10 - 6 = 32$.
4-way set-associative. The number of sets is $1024/4 = 256$, so index bits = $\log_2 256 = 8$. Tag bits = $48 - 8 - 6 = 34$.

This example captures a general principle: increasing associativity reduces the number of sets, which reduces index bits and increases tag bits.

5.8 The 3Cs of Cache Misses

Misses are traditionally classified into three categories.

- **Compulsory (cold) misses**: the first reference to a block must miss because the cache has not seen the block before.
- **Capacity misses**: the working set is larger than the cache can hold, so blocks are evicted and later needed again.
- **Conflict misses**: the cache has enough total space, but placement restrictions force useful blocks to collide in the same set or line.

This decomposition is more than terminology. It guides optimization. Larger blocks can reduce compulsory misses. Larger capacity targets capacity misses. Greater associativity targets conflict misses.

In multiprocessors, some analysts also identify *coherence misses* caused by invalidations from other cores. The classic 3C model remains the right starting point for single-core reasoning, but modern shared-memory systems often require the extra coherence perspective.

6 Replacement and Write Policies

6.1 Replacement: Which Block Becomes the Victim?

When a miss occurs in a set that has no free entry, the cache must choose a *victim*. In a direct-mapped cache the choice is forced, but in associative caches a replacement policy is required.

Typical policies include:

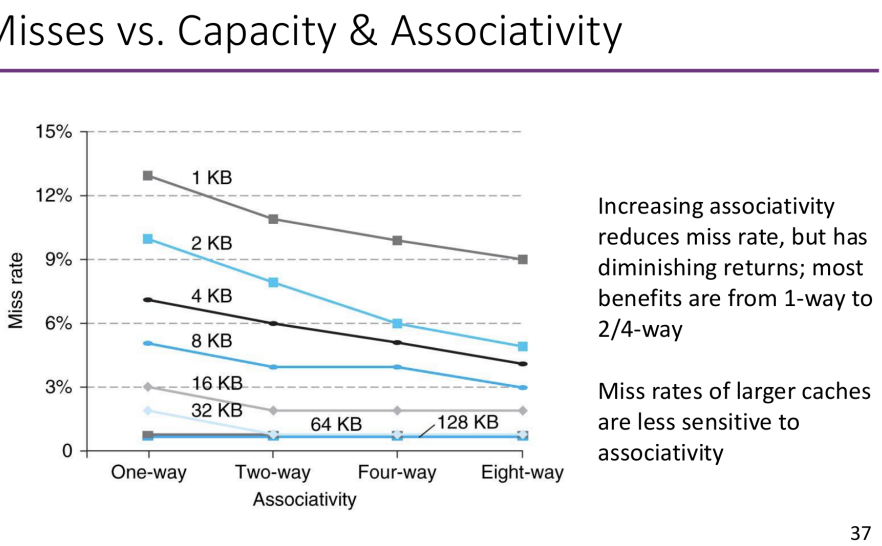


Figure 8: Miss rate as a function of cache capacity and associativity. The figure shows two enduring patterns: larger caches reduce misses overall, and increasing associativity reduces conflict misses but with diminishing returns. In practice, moving from direct-mapped to 2-way or 4-way often captures most of the benefit.

- **Random**: choose a way at random. Surprisingly competitive in some workloads because it avoids metadata complexity.
- **LRU (Least Recently Used)**: evict the block that has gone unused for the longest time, based on the assumption that recently used data are likely to be used again.
- **LFU (Least Frequently Used)**: prefer to keep blocks with a history of frequent reuse.

LRU is conceptually attractive because it aligns with temporal locality, but exact LRU can be expensive to implement for high associativity. Real processors often approximate it with pseudo-LRU or related schemes.

Deep Dive

LRU is not always optimal. A streaming access pattern whose working set is slightly larger than the cache can make LRU thrash badly: every newly fetched block evicts the one that will be needed next. This is a reminder that a "reasonable" policy can still be defeated when the access pattern systematically exceeds capacity.

6.2 Bypass and Insertion Choices

A miss does not always imply that the requested block should enter the cache. In some situations, especially for streaming or one-time accesses, inserting the block pollutes the cache and harms future reuse. Some designs therefore allow *bypass*, where the block is forwarded to the requester without being installed in the cache. Modern processors may use dynamic insertion policies to decide how aggressively a missed block should be admitted and how long it should be retained.

6.3 Write Hits and Write Misses

Reads are conceptually simpler than writes because a read only consumes data while a write changes system state. A data cache therefore needs an explicit write policy.

6.3.1 Write-Back versus Write-Through

Under **write-through**, a write updates both the cache and the next lower level immediately. This simplifies consistency between levels but increases bandwidth demand.

Under **write-back**, a write updates only the cache on a hit. The modified block is written to the next lower level later, typically when it is evicted. To support this behavior, each line stores a **dirty bit** indicating whether its cached contents differ from the lower level.

Most performance-oriented processor data caches use write-back because it absorbs repeated writes to the same block cheaply and reduces lower-level traffic.

6.3.2 Write-Allocate versus No-Write-Allocate

On a write miss, a cache may either fetch the block into the cache and then perform the write (**write-allocate**), or write the data directly to the next level without bringing the block in (**no-write-allocate**, sometimes called write-no-allocate).

The most common high-performance combination is **write-back + write-allocate**. It assumes that if a program writes one part of a block, it may write or read nearby parts again soon, so installing the block is worthwhile.

Deep Dive

Why fetch an entire block on a write miss even if a single word is being overwritten? There are several reasons. First, many writes do not overwrite the entire line, so the untouched bytes must be preserved. Second, nearby bytes may soon be read or written, so fetching the full block exploits spatial locality. Third, coherence and lower-level interfaces are usually line-based rather than byte-based, so line-granularity handling keeps the hierarchy conceptually consistent.

6.4 The "ABC" View of Cache Configuration

A concise way to remember core cache design parameters is to focus on:

- **Associativity** A,
- **Block size** B,
- **Capacity** C.

These parameters determine both the address decomposition and the dominant miss mechanisms. In summary:

- Increasing associativity mainly fights **conflict** misses.
- Increasing block size mainly reduces **compulsory** misses, until wasted bandwidth becomes a problem.

- Increasing capacity mainly reduces **capacity** misses.

7 Cache-Friendly Programming and Performance Optimization

7.1 Caches Are Functionally Transparent but Performance Visible

A cache does not change the meaning of a correct load or store. Software uses the same addresses regardless of whether the data are found in L1, L2, L3, or main memory. This is why cache configurations usually belong to microarchitecture rather than to the instruction-set architecture.

Performance is different. The same source program can run much faster or much slower depending on how its access pattern interacts with the hierarchy. This is one of the most important messages of the lecture: *good systems code is locality-aware code.*

7.2 The Memory Mountain

The so-called *memory mountain* is a visualization of throughput as a function of working-set size and access stride. It condenses an enormous amount of architectural knowledge into one surface plot.

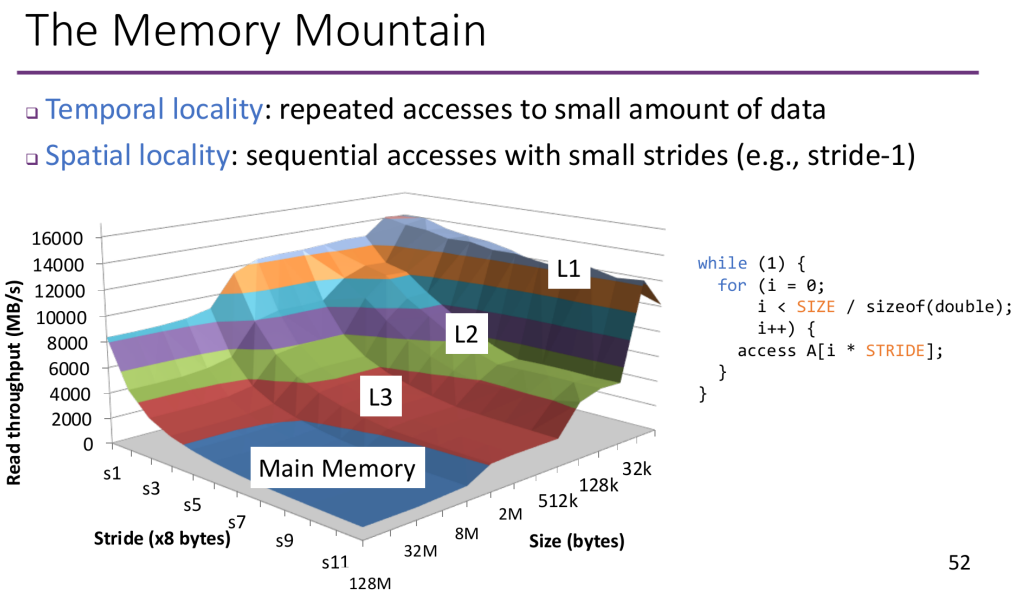


Figure 9: The memory mountain. Throughput is highest when the working set fits in upper cache levels and accesses have small stride, thereby exploiting both temporal and spatial locality. As the working set exceeds L1, L2, L3, and finally main memory, performance drops in steps. Larger strides waste cache-line payload and reduce the effectiveness of hardware prefetching.

Several lessons follow immediately.

First, a small working set improves temporal locality because recently accessed lines are likely to remain resident. Second, stride-1 or small-stride traversals exploit spatial locality by using most of each fetched line. Third, performance cliffs often correspond to specific capacity boundaries: when the working set no longer fits in L1, the access stream falls to the L2 plateau, and so forth.

7.3 Arrays, Row-Major Layout, and Stride

In C and C++, multi-dimensional arrays are stored in *row-major* order. For an array `a[M][N]`, the address of `a[i][j]` is proportional to

$$(i \cdot N + j) \times \text{sizeof(element)}.$$

Thus varying j in the innermost loop walks through contiguous memory, while varying i in the innermost loop jumps by an entire row length each time. The former exhibits good spatial locality; the latter often does not.

This one fact explains why some loop nests that are mathematically equivalent differ drastically in runtime.

7.4 A Simple Example: Summing a Matrix

Consider the following code, which traverses a two-dimensional array column by column rather than row by row.

Listing 1: A matrix traversal with poor spatial locality in C row-major storage.

```
int sum(int a[M][N]) {
    int s = 0;
    for (int j = 0; j < N; j++)
        for (int i = 0; i < M; i++)
            s += a[i][j];
    return s;
}
```

If the language stores rows contiguously, the inner loop above steps through memory with a large stride. A locality-aware version simply swaps the loop order so that the inner loop advances through contiguous columns of a single row.

Listing 2: A cache-friendly traversal of the same matrix.

```
int sum(int a[M][N]) {
    int s = 0;
    for (int i = 0; i < M; i++)
        for (int j = 0; j < N; j++)
            s += a[i][j];
    return s;
}
```

The arithmetic is identical; the access pattern is not.

7.5 Matrix Multiplication as a Case Study

Matrix multiplication is the canonical locality example because it performs many operations on regular data and admits several mathematically equivalent loop orders. Consider the naive triple loop for multiplying $C = C + A \times B$.

Listing 3: The classical ijk order for matrix multiplication.

```
for (int i = 0; i < n; i++)
```

```
    for (int j = 0; j < n; j++)
        for (int k = 0; k < n; k++)
            c[i][j] += a[i][k] * b[k][j];
```

The arithmetic complexity is $O(n^3)$ no matter how the loops are permuted. The cache behavior, however, depends entirely on which index varies fastest.

In row-major storage, the expression `a[i][k]` has good spatial locality when k is the innermost index because elements of a row are contiguous. The expression `b[k][j]` is problematic in the same order because changing k with fixed j walks down a column of `b`, which is strided in memory. Finally, `c[i][j]` can enjoy temporal locality if the same destination element is repeatedly updated before moving on.

7.6 Loop Reordering

Different loop orders privilege different matrices. The lecture slides compare the six permutations and show that orders such as `kij` and `ikj` often outperform `ijk`, `jik`, `jki`, and `kji` on row-major machines.

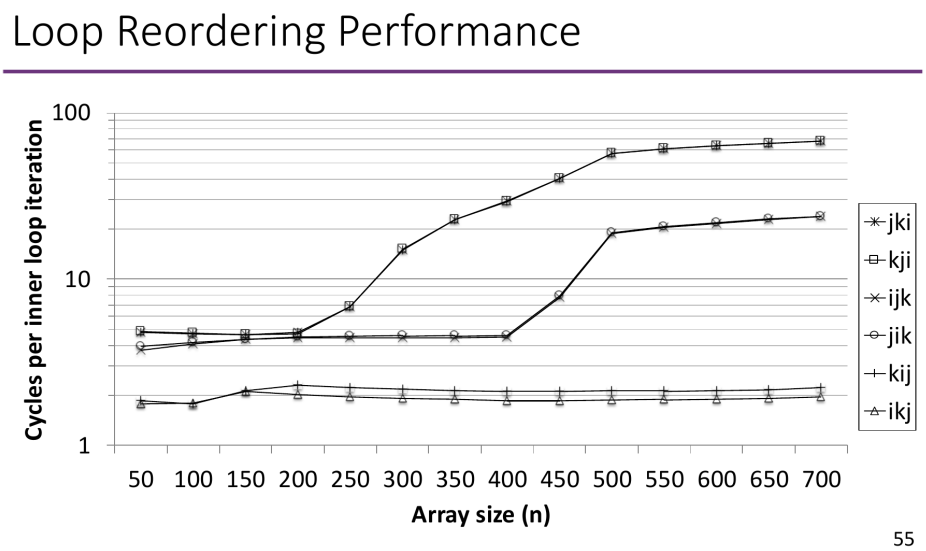


Figure 10: Measured performance for different matrix-multiplication loop orders. Although all versions perform the same abstract arithmetic, their cycles per inner-loop iteration differ greatly because their memory-access patterns interact differently with the cache hierarchy. This is a vivid example of microarchitectural performance sensitivity beneath identical asymptotic complexity.

Why are `kij` and `ikj` attractive? Because they allow `b[k][j]` and `c[i][j]` to be traversed with good spatial locality in the inner loop over j, while also reusing `a[i][k]` across many updates. In contrast, orders that make i the innermost loop often suffer badly because both `a[i][k]` and `c[i][j]` become strided.

7.7 Loop Blocking (Tiling)

Even the best simple loop order cannot make three large matrices simultaneously fit in a small cache. Blocking, also called tiling, addresses this by partitioning the computation into submatrices small enough that a few working tiles remain resident while many arithmetic operations are performed on them.

Algorithm 1: Blocked Matrix Multiplication

Input: Matrices A, B, and C of size $n \times n$; block size B_s
Output: Updated matrix $C = C + A \times B$

for $kk \leftarrow 0$; $kk < n$; $kk \leftarrow kk + B_s$ **do**
  for $ii \leftarrow 0$; $ii < n$; $ii \leftarrow ii + B_s$ **do**
    for $jj \leftarrow 0$; $jj < n$; $jj \leftarrow jj + B_s$ **do**
      for $k \leftarrow kk$ **to** $\min(kk + B_s - 1, n - 1)$ **do**
        for $i \leftarrow ii$ **to** $\min(ii + B_s - 1, n - 1)$ **do**
          for $j \leftarrow jj$ **to** $\min(jj + B_s - 1, n - 1)$ **do**
            $C[i][j] \leftarrow C[i][j] + A[i][k] \times B[k][j]$;
return C;

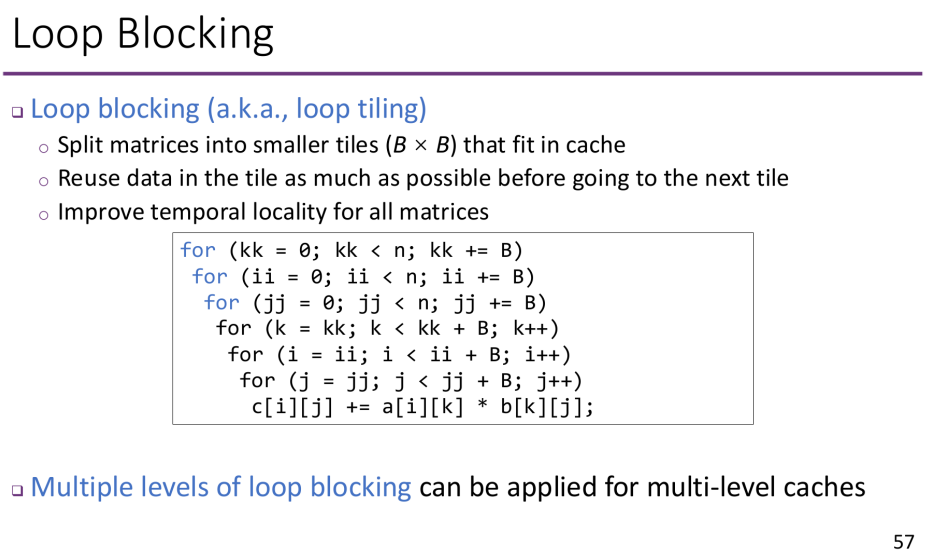


Figure 11: Loop blocking for matrix multiplication. The matrices are divided into tiles so that a small number of $B \times B$ submatrices fit in cache simultaneously. By performing many arithmetic operations before moving to the next tile, the program increases temporal reuse for all three matrices rather than relying on favorable locality for only one of them.

The key design rule is to choose the largest tile size that keeps the relevant blocks of A, B, and C in cache simultaneously. If the cache can effectively hold three tiles and the element size is e bytes, then a rough condition is

$$3 \times B_s^2 \times e \lesssim C_{\text{usable}}.$$

This formula is approximate because real caches have metadata overhead, associativity limits, competing traffic, and instruction footprint, but it captures the underlying principle.

7.8 Why Blocking Works

Without blocking, each row or column element may be loaded repeatedly from lower levels because the total working set is too large. With blocking, a fetched tile is reused many times before eviction. The total number of misses can therefore drop by roughly a factor proportional to the block size in idealized analyses.

This illustrates a broader lesson: performance optimization often means reshaping the schedule of computation so that reuse occurs while the data are still resident in a fast level of the hierarchy.

7.9 Compiler and Library Support

In high-performance numerical software, locality optimization is rarely left to naive source code. Libraries such as BLAS and vendor-tuned math kernels use careful loop order, blocking, vectorization, register tiling, prefetching, and instruction scheduling. Systems such as Halide and TVM expose loop transformations explicitly and may search large optimization spaces automatically.

This is one reason performance engineering can be processor-dependent. A transformation that is ideal for one hierarchy may be merely acceptable for another. Nevertheless, the underlying conceptual tools—locality, working sets, stride, blocking, and reuse—remain stable across implementations.

8 Summary

Memory hierarchy is the systems answer to a physical tradeoff. SRAM is fast but small. DRAM is large but slower. Secondary storage is larger still but much slower, while offering persistence. Because programs tend to exhibit temporal and spatial locality, a system can keep the currently useful subset of code and data close to the processor in fast memory and thereby deliver performance far better than a flat large memory would allow.

Caches operationalize this idea. They move blocks, distinguish hits from misses, and balance hit latency against miss rate through choices such as associativity, block size, capacity, replacement policy, and write policy. Average Memory Access Time provides a compact way to reason about these tradeoffs. Multi-level caches refine the same idea across several latency bands, and multi-core systems add coherence as a correctness requirement.

From a programmer's perspective, the lecture delivers an equally important lesson: caches are mostly invisible to correctness but highly visible to performance. Code that walks arrays sequentially, reuses data soon after loading, and blocks large computations into cache-sized tiles will often outperform code with the same arithmetic complexity but poor locality. Understanding the hierarchy is therefore not merely a hardware topic; it is an essential programming skill.

9 Quizzes & Exercises

9.1 Review Questions

1. Explain why primary memory and secondary storage serve different purposes in a computer system.

2. Why is SRAM used for caches while DRAM is used for main memory?
3. Define temporal locality and spatial locality, and give one example of each from code.
4. What is a cache block or cache line, and why is data transferred at block granularity rather than byte granularity?
5. Define hit rate, miss rate, hit latency, miss penalty, and AMAT.
6. Contrast direct-mapped, fully associative, and set-associative caches in terms of hit latency and miss rate.
7. Why does a cache need both a tag and a valid bit?
8. Distinguish compulsory, capacity, and conflict misses.
9. Why can a 99% hit rate be much more than "2% better" than a 97% hit rate?
10. What problem does cache coherence solve in a multi-core processor?
11. What is false sharing, and why can it degrade performance even when threads do not intentionally share variables?
12. Why can two matrix-multiplication loop orders with the same $O(n^3)$ arithmetic complexity have very different runtimes?

9.2 Analytical Exercises

1. A cache has hit latency 1 ns, miss rate 4%, and miss penalty 80 ns. Compute the AMAT.
2. A 64 KiB cache uses 64-byte blocks and is 4-way set-associative. Assuming 48-bit addresses, compute the number of sets and the number of tag, index, and offset bits.
3. Give one access pattern that is dominated by compulsory misses, one dominated by conflict misses, and one dominated by capacity misses.
4. Consider an array `double a[M][N]` stored in C row-major order. Which loop order is more cache-friendly for visiting all elements: outer loop over `i` and inner loop over `j`, or the reverse? Explain using the notion of stride.
5. In a write-back cache, what additional metadata is needed beyond tag and valid bit? Why?
6. Suppose two cores repeatedly increment two distinct counters that accidentally occupy the same cache line. Explain qualitatively what happens in the coherence protocol and why throughput may collapse.
7. For matrix multiplication, explain why blocking improves temporal locality for all three matrices rather than for only one.
8. A pointer-chasing graph traversal has poor spatial locality. Suggest two practical techniques that could still improve performance without changing the high-level algorithmic goal.

9.3 Programming-Oriented Exercises

1. Rewrite the following code to improve locality, and justify your answer:

```
for (int j = 0; j < N; j++)
    for (int i = 0; i < M; i++)
        s += a[i][j];
```

2. Implement two versions of matrix multiplication: `ijk` and `kij`. Measure runtime for increasing matrix sizes and explain the trend using cache behavior.
3. Add loop blocking to your fastest matrix multiplication variant. Experiment with several tile sizes and relate the best-performing value to your processor's cache sizes.
4. Write a microbenchmark that reads an array with stride 1, 2, 4, 8, and 16 cache-line words. Plot throughput and explain the shape using the memory mountain idea.

9.4 Optional Preview Questions from the Uploaded Auxiliary Guide

The uploaded auxiliary study guide primarily targets the next lecture on processes and threads rather than the present lecture on memory hierarchy. To preserve topical coherence, those items are placed here only as an optional preview.

1. What is a process, and how does it differ from a program?
2. What are the two major abstractions provided by a process?
3. What is the function of a Process Control Block (PCB)?
4. Compare user mode and kernel mode in terms of privilege.
5. Why do terminated processes become zombies, and how are they eventually removed?
6. Essay prompt: analyze the tradeoff between protection and efficiency when threads share an address space but processes do not.